

LibTW FAQ

I want to ...

- ... [set up LibTW.](#)
 - ... [calculate a tree decomposition of my graph.](#)
 - ... [calculate the treewidth of my graph.](#)
 - ... [implement a new algorithm.](#)
 - ... [draw my graph.](#)
 - ... [easily do time measurements.](#)
 - ... [run regression testing.](#)
 - ... [add regression tests for an existing algorithm.](#)
 - ... [add an algorithm to the regression testing.](#)
-

... set up LibTW.

First of all, get the jar file or the source. If you are going to develop LibTW or are curious, you can get the source from [link TODO]. If you will just use it as a library, you can get the jar file from [link TODO]. The javadocs can be built from the source, or found online [link TODO]. Note that LibTW requires Java 1.5: it will not compile on earlier versions because it uses generics.

Either way, all the LibTW classes are in the `nl.uu.cs.treewidth` package. If everything is alright, you can run the following demo application. Do this from the project root.

```
java nl.uu.cs.treewidth.Main
```

This should produce output like the following.

```
LibTW V1.0
```

```
This library is free software; you can redistribute it and/or
modify it under the terms of the GNU Lesser General Public
License as published by the Free Software Foundation
```

```
***** Warning *****
You have loaded a multigraph. Duplicate edges have been removed!
*****
```

```
Graph           : queen5_5.dgf
# Vertices in graph : 25
# Edges in graph   : 160
```

```
Permutation algorithm : Lex-BFS: Lexicographic Breadth First Search
Upperbound            : 18
Time needed           : 133 ms
```

```
Upperbound algorithm : GreedyDegree
Upperbound            : 18
Time needed           : 9 ms
```

```
Lowerbound algorithm : MinDegree
Lowerbound            : 12
Time needed           : 0 ms
```

```
Exact algorithm : QuickBB aka KwikBieBieDrieieie
Treewidth       : 18
Time needed      : 19097 ms
```

```
Exact algorithm : Treewidth Dynamic Programming algorithm with upperbound and clique
Treewidth       : 18
Time needed      : 305 ms
```

Done.

Note that this can take a while to run: the LibTW library implements two exact algorithms for treewidth, and this demo program runs them both on the 5 by 5 queen graph. (The run quoted above took about 20 seconds on a 1 GHz machine.) See the other entries of this faq for pointers on how to use LibTW as a library and how to develop it further.

... calculate a tree decomposition of my graph.

The following code will calculate and display a tree decomposition. The code demonstrates typical use of the LibTW library.

In this example, we'll load a graph from a DGF file. We will ignore possible file IO exceptions.

NGraph is the standard graph data structure in LibTW. It is generic in the type of data that its vertices contain. InputData is most common here: it is what all the classes in the input package return, and what all Algorithms expect to get.

```
NGraph<InputData> g = null;

GraphInput input = new DgfReader( "myGraph.dgf" );
try {
    g = input.get();
} catch( InputException e ) {}
```

We have now loaded a graph. First of all, lets calculate a lowerbound and an upperbound. This is a good idea for several reasons. First of all, we might get lucky and find the treewidth this way, if the lowerbound equals the upperbound. Otherwise we will have to use another algorithm, but the one we will be using wants an upperbound anyway. (Also, these algorithms are quite fast.) The heuristics we use here are the typically the best to use.

Calling an algorithm takes four steps.

1. Create an instance of a class that implements Algorithm, or more specifically, one of Algorithm's sub-interfaces (e.g. LowerBound, UpperBound).
2. Tell it what to work on using setInput.
3. Tell it to do its thing using run.
4. Get the result (e.g. getLowerBound, getUpperBound).

You can see this in action here. (Note that the algorithms themselves are also generic. This is the type of data they want the vertices to contain. Remember our graph has InputData in its vertices.)

```
MaximumMinimumDegreePlusLeastC<InputData> lbAlgo = new MaximumMinimumDegreePlusLeastC<InputData>();
lbAlgo.setInput( g );
lbAlgo.run();
int lowerbound = lbAlgo.getLowerBound();

GreedyFillIn<InputData> ubAlgo = new GreedyFillIn<InputData>();
ubAlgo.setInput( g );
ubAlgo.run();
int upperbound = ubAlgo.getUpperBound();
```

Now we have a lowerbound and an upperbound. If they are equal, we now know the treewidth, but we wanted a tree decomposition. Luckily the GreedyFillIn algorithm works by heuristically determining a permutation and returning the treewidth induced by that permutation, and we can ask it for that permutation. (Most upperbounds work this way.) Permutations of vertices are represented by the NVertexOrder class.

```
NVertexOrder<InputData> permutation = null;

if( lowerbound == upperbound ) {
    permutation = ubAlgo.getPermutation();
} else ...
```

We will turn this permutation into a tree decomposition later; first we will look at the the case where we weren't so lucky and our lowerbound and upperbound heuristics don't agree. We will bring on one of the big guns: the QuickBB algorithm. This is an exact algorithm which will find a permutation with optimal treewidth.

```
... else {
    QuickBB<InputData> qbbAlgo = new QuickBB<InputData>();
    qbbAlgo.setInput( g );
    qbbAlgo.run();
    permutation = qbbAlgo.getPermutation();
}
```

Now we have a permutation with optimal treewidth. (In the latter case because QuickBB is an exact algorithm, in the former because we have a lowerbound that tells us our upperbound is actually optimal.) All we have left to do is calculate the corresponding tree decomposition. This last step is also as simple as calling an algorithm. We tell it which permutation to use (ours!) in the constructor. Note that it still needs to know about our graph.

So far everything has been rather simple. The following is maybe the most complicated-looking part of the code: the type of the tree decomposition. It is a graph just like any other, only the vertices contain sets of vertices of the original graph. So there is a class to hold sets of vertices and it is called NTDBag, for Tree Decomposition Bag. We want it to be sets of vertices from our original graph, which had InputData as data. So in total we have: an NGraph of NTDBags of InputData.

```
PermutationToTreeDecomposition<InputData> convertor = new PermutationToTreeDecomposition<InputData>( p );
convertor.setInput( g );
convertor.run();
NGraph<NTDBag<InputData>> decomposition = convertor.getDecomposition();
```

We now have a tree decomposition of g! Lets have a look.

```
decomposition.printGraph( true, true );
```

This shows the tree decomposition in a window and writes it to a PNG in the current directory. (You need to have GraphViz installed correctly for this. See: [I want to draw my graph.](#))

... calculate the treewidth of my graph.

If you don't need the actual tree decomposition, the TreewidthDP algorithm is a lot faster than the other exact algorithm, QuickBB. Just like when you are calculating a tree decomposition, it is a good idea to first calculate a lower and upperbound. (For details, see: [I want to calculate a tree decomposition of my graph.](#))

Next, use the TreewidthDP like any other algorithm, but don't forget to tell it about the upperbound you've just found: this will speed up the algorithm a lot.

```
int upperbound = ...an upperbound...
TreewidthDP<InputData> twdp = new TreewidthDP<InputData>( upperbound );
twdp.setInput( g );
twdp.run();
int treewidth = twdp.getTreewidth();
```

TreewidthDP uses a lot of memory. The standard Sun virtual machine limits the Java heap memory to 64 MB: if you use more, you will get an out-of-memory exception. This might not be enough for TreewidthDP. To increase the amount of memory your java application is allowed to use in the standard Sun virtual machine, set the -Xmx flag. The following command allows for 800 MB of heap space, which was about right for our machines with 1 GB of RAM to not start using virtual memory.

```
java -Xmx800M MyClassWhichUsesTreewidthDP
```

... implement a new algorithm.

As an example, we'll implement a trivial upperbound: the number of vertices minus one. All algorithms in LibTW are classes that implement Algorithm, and in particular, one of its sub-interfaces. Also, all algorithms are generic in the data in the graph they are going to work on. Your algorithm will have to be of the following format: a single generic parameter which extends InputData (that's what your algorithm can count on: you'll never get less than InputData in the graph) and it implements an Algorithm sub-interface.

```
public class NumVertices< D extends InputData > implements UpperBound<InputData> {
```

Next, we will need some simple methods and fields. We have a field to hold our upperbound. The run method will give it the intended value, but getUpperBound can be called before run. That's why we initialize it to "plus infinity" in the constructor: this way, getUpperBound will always return a valid upperbound. The getName method is unexciting, but required. The setInput method needs a little extra explanation: here we just keep a reference to the graph. That's okay in this algorithm because we are not going to change the graph, but if your algorithm changes the graph in any way, make a deep copy of it. (See the NGraph javadocs on how to do that.) If you want to convert the graph to a more convenient format, this is also the place to do that. The algorithm itself is particularly simple.

```
protected NGraph graph;
protected int upperbound;

public NumVertices() {
    upperbound = Integer.MAX_VALUE;
}

public String getName() {
    return "Number of vertices minus one.";
}

public void setInput( NGraph g ) {
    graph = g;
}

public int getUpperBound() {
    return upperbound;
}

public void run() {
    upperbound = graph.getNumberOfVertices();
}

}
```

... draw my graph.

Install GraphViz (available online for free). Make sure the ``dot'` and ``neato'` commands are available directly from the command prompt (i.e.: add them to the PATH if the installer didn't do this for you). Then, the following simple code is all you need, where g as an NGraph.

```
g.printGraph( true, true );
```

The first boolean indicates whether the graph should be shown in a window, the second whether the graph should be written as a PNG file. For more options, look at the `Output.present(...)` set of methods.

... easily do time measurements.

For this we have the convenient class `Stopwatch`, in the timing package. The simplest way to use it is as follows.

```
Stopwatch t = new Stopwatch();
t.start();

// ... do stuff ...

t.stop();
long millisecondsPassed = t.getTime();
```

You can also start it again, stop it again, reset it, etc. One thing to realize is that its precision is only about 20 ms. If you need more precision, you can do this as follows.

```
Stopwatch t = new Stopwatch( new JavaNanoTime() );
```

You should be careful with this more precise timer. First of all, it is not guaranteed to work correctly on multiprocessor / dual-core machines. It is safe in most situations, but do read the documentation if you want to use it. Secondly, you will probably want to change your experiment instead if you need this much precision.

... run regression testing.

The regression testing is done by running the ResultChecker, which is in the testing package. Run it from your IDE of choice, or from the command line as follows. (You should be in the project root.)

```
java nl.uu.cs.treewidth.testing.ResultChecker
```

This reads the file default.tests from the current directory to see which tests to run.

It asks for input on any failed test. This may not be what you want in an automated system. If you want anything more sophisticated, feel free to add it. (But do remember that LibTW is LGPL, and we would like to hear about your improvements.)

... add regression tests for an existing algorithm.

The ResultChecker is used for regression testing and it reads test descriptions from file. The current version only reads the default.tests file in the current directory. (You can easily change this in the source.) The format of the .tests file is very simple. A test description is on one line, with five elements separated by tabs.

1. The type of algorithm: lowerbound, upperbound or exact. Only algorithms with a numeric answer are supported.
2. The name of this algorithm. Note that this is not the classname of the algorithm, but a name that has been hardcoded into the ResultChecker. (To add your own algorithm, see: I want to add an algorithm to the regression testing.)
3. The filename of the graph to load.
4. The correct answer.
5. Comments. If the test fails, this message will be presented. It is very important to explain here why you believe the answer you specify in this test is in fact correct. It should convince the person who encounters a supposed bug that his code is wrong, and not this test. (For most of the existing tests, we refer to a TreewidthLIB URL, a well known repository of experimental results.) Note that, while you can use spaces here, you cannot use tabs here due to the simple implementation of ResultChecker.

As an example, let's look at one of the existing tests: the GreedyDegree algorithm should give upperbound 10 when run on the "celar02" graph.

upperbound	GreedyDegree	graphs/celar02.dgf	10	http://www.cs.uu.nl/~hansb/treewidthlib/resultt
------------	--------------	--------------------	----	---

... add an algorithm to the regression testing.

This involves adding some code to the ResultChecker. But you've been writing code anyway, so this won't be a problem. As an example, we'll add a MyOwnUpperbound algorithm. The only method we will change is makeUBCreators. This makes a map from strings to an object which will create the appropriate algorithm. This involves an anonymous class and may look complicated, but to just add your own algorithm, a copy-paste job will do.

```
map.put( "MyOwnUpperbound",
        new AlgoCreator() { public UpperBound<InputData> create() {
            return new MyOwnUpperbound<InputData>();
        } }
    );
```

Adding a new type of algorithm (besides lowerbound, upperbound, exact) is slightly more complicated, but you should be able to figure it out from reading the code.